Colour Maximite 2
C Compiler

Compiling C and MMBasic programs on the Colour Maximite 2

For firmware 7.00.00RC2 and later

Compiler: the Fuzix Compiler Kit
Port and guide: Peter Mather

(10 November 2026)

The firmware, the compiler and this guide are at
https://github.com/UKTailwind/CMM2V1.5

About This Guide

This guide is for users of the CC command, which compiles programs written in C, and MMBasic programs, on the Colour Maximite 2 itself. It covers installing the compiler, compiling and running a program, the language and library, what is and is not translated from MMBasic, and the speed to expect. It assumes you can use the MMBasic command prompt and copy files to the SD card.

Support

Questions should be raised on the Back Shed forum (http://www.thebackshed.com/forum/Microcontrollers), and problems with the compiler reported as issues of the repository above.

Copyright

This guide is copyright 2026 Peter Mather and is distributed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Australia license (CC BY-NC-SA 3.0), as the user manual is. The compiler is the Fuzix Compiler Kit, under the GNU General Public License version 3; its other parts have their own licences (see Appendix A).

Contents

Introduction 4
When it is worth compiling 4
What you need 4
Where it comes from 4
Getting Started 5
Installing it 5
Your first C program 5
Your first compiled BASIC program 5
CC's options 5
C Programs 6
The language 6
The library 6
Graphics, sound and the rest 6
How a program runs 6
MMBasic Programs 7
What it does with your program 7
What it translates 7

What it does not translate 7
Checking the result 7
Very large programs 7
Speed 8
Calculation 8
Games 8
Which to use 8
Problems 9
CC says it cannot find the compiler 9
"Cannot open include file stdint.h" 9
"x.c did not compile" 9
A compiled BASIC program does the wrong thing 9
A game runs too fast 9
"not enough memory for its tables", "jump out of range", "code segment full" 9
CC seems to be taking a long time 9
The compiled program cannot find its files 9
Appendix A – Licences 10

Introduction

The Colour Maximite 2 can compile programs on the machine itself, with no PC. The CC command compiles a program written in C, and it also compiles an MMBasic program, so that a game or a calculation you already have can run faster with no change to it.

```
CC "hello.c"
RUN "hello.bc"

CC "game.bas"
RUN "game.b.bc"
```

Your BASIC program is not changed. RUN "game.bas" still runs it with the interpreter as it always did, and the compiled copy sits beside it in a file of its own.

When it is worth compiling

Your program	What to use
An MMBasic program that is fast enough	Nothing: leave it as it is.
An MMBasic program that spends its time calculating	CC. It is much the fastest way to run it.
An MMBasic game that spends its time drawing	Try OPTION COMPILE ON first. CC gains less here, as the drawing takes the same time however the program is run.
Something new that has to be as fast as it can be	Write it in C, with CC, or as a loadable program on a PC.

The chapter Speed has the figures.

What you need

- A Colour Maximite 2, Generation 1 or 2, with firmware 7.00.00RC2 or later.
- The cc folder of CMM2_Programs_7.00.00RC6.zip, copied to the SD card as A:/cc.

The firmware holds the part that runs a compiled program, so a compiled program needs nothing from the card to run. A:/cc is needed only to compile.

Where it comes from

The compiler is the Fuzix Compiler Kit, with the bytecode and Arm (Thumb-2) code generator written for the Pico Computer 3. The translator from MMBasic to C was written for the same machine. Both were brought to the Colour Maximite 2 by Peter Mather. The Appendix gives their licences and where the source is.

Getting Started

Installing it

Copy the cc folder from CMM2_Programs_7.00.00RC6.zip to the SD card, so that the card has:

File	What it is
A:/cc/cc.app	The compiler.
A:/cc/include	The header files that C programs include.
A:/cc/examples	Two example programs, shapes.c and blink.c.

It has to be in A:/cc. CC looks for it there, whatever the current directory is.

Your first C program

Type this into a file called hello.c, with EDIT or on a PC:

```
#include <stdio.h>

int main(void)
{
    printf("Hello from C, %d\n", 6 * 7);
    return 0;
}
```

Then, in the same directory:

```
CC "hello.c"
RUN "hello.bc"
```

CC makes hello.bc beside hello.c. It takes about a second, and prints a row of dots as it works. RUN then runs it, and MMBasic carries on when the program ends. Ctrl-C stops a program, as it does in BASIC.

RUN can do both steps. RUN "hello.c" compiles the file if hello.bc is missing or older than it, and then runs it, so that editing the program and trying it again is one step.

Words after a comma are the program's arguments, in argv[1] onward:

```
RUN "hello.bc", one two
```

Your first compiled BASIC program

Any MMBasic program can be tried. CHDIR to its folder first, as a game finds its pictures and sounds from there:

```
CHDIR "A:/game"
CC "game.bas"
RUN "game.b.bc"
```

CC translates the program to C, in game.mb.c, and compiles that to game.b.bc. It prints "wrote game.mb.c" and then the dots. Any line it could not translate is reported as it goes (see the chapter MMBasic Programs).

A program of about 3,000 lines compiles in 7 seconds. The largest we have tried, 4,300 lines, takes about a minute and a half.

CC's options

CC takes a second argument, a string of options for the compiler:

Option	What it does
-v	Shows each pass of the compiler as it runs.
-k	Keeps the intermediate files.
-o name	Gives the object file another name.

For example:

```
CC "hello.c", "-o greet.bc"
```

C Programs

The language

The language is C89 (ANSI C) with long long, // comments and declarations after statements. A program is one source file, as there is no linker.

These are not supported:

- bit fields;
- declarations inside a for statement, as in for(int i = 0; ...): declare i before it;
- local arrays initialised from a string: "static char s[] = ..." works, and "char s[] = ..." inside a function does not.

Errors are listed with the file's name and the line number (hello.c:12 - missing semicolon), and CC then gives the error "hello.c did not compile".

The library

- stdio.h: printf, sprintf and fprintf, putchar and puts, and files with fopen, fread, fwrite, fgetc, fgets, fputc, fputs, fseek, ftell, rewind, feof, fflush, fclose, remove and rename. There is no scanf.
- stdlib.h: malloc, calloc, realloc and free, atoi, atol, atof, strtod, strtol and strtoul, abs, labs and llabs, rand and srand, and exit.
- string.h: strlen, strcpy, strncpy, strcat, strcmp, strncmp, strchr, strrchr, memset, memcpy, memmove and memcmp.
- math.h: sin, cos, tan, asin, acos, atan, atan2, sinh, cosh, tanh, sqrt, exp, log, log10, pow, floor, ceil, fabs and fmod, in double precision.
- ctype.h: isdigit, isalpha and the others, toupper and tolower.
- time.h: time, clock (CLOCKS_PER_SEC is 1000000), and time_us and time_us64, the microseconds since the Colour Maximite 2 started.
- cmm2.h: the Colour Maximite 2's own facilities (below).

Files have MMBasic's names: A:/dir/name.txt, or a name relative to the current directory.

Graphics, sound and the rest

cmm2.h gives a C program the services that the firmware gives a loadable program, by the same names: the video modes and the palette, drawing with MMBasic's own BOX, CIRCLE, TEXT and the rest (gfx_box, gfx_circle, gfx_text...), saving the screen as a PNG, the keyboard, the mouse, sound, and the Colour Maximite 2's pins. The companion manual *Colour Maximite 2 Loadable Programs* describes each service.

```
#include <cmm2.h>

int main(void)
{
    cmm2_video v;

    video_mode(1, 8, &v);
    gfx_cls(BLACK);
    gfx_circle(400, 300, 100, 2, 1.0, GREEN, RED);
    gfx_text(400, 300, "Hello", "CM", 1, 2, WHITE, NOFILL);
    return 0;
}
```

The screen is left as it is when the program ends, and MMBasic's own mode and palette come back when it does. The framebuffer that video_mode gives is an ordinary pointer, so a program can also draw by writing to it.

A:/cc/examples/shapes.c draws with these services, and blink.c flashes an output pin.

How a program runs

Each function is translated to the processor's own instructions where the compiler can, and the rest to a compact bytecode that the firmware interprets. Floating point uses the processor's double precision hardware. A program takes about 16ms to start.

MMBasic Programs

CC translates an MMBasic program to C and compiles the C. This is not OPTION COMPILE, which speeds up the interpreter as it runs: CC makes a separate compiled program, and needs no OPTION.

What it does with your program

- game.bas is read, and any #INCLUDE files are read in as MMBasic would.
- game.mb.c is written: the translation, in C. CC removes it afterwards unless you give it the option -k.
- game.b.bc is written: the compiled program. Run this with RUN "game.b.bc".

If you change game.bas, run CC again. RUN "game.b.bc" does not check that it is up to date.

What it translates

Almost all of MMBasic is translated into C: the arithmetic, strings, arrays, structures, SUBs and FUNCTIONs, files, the drawing commands, and DATA and SORT.

The Colour Maximite 2's larger graphics and sound commands are handed to the firmware to run: PAGE, IMAGE, MODE, BLIT, LOAD PNG, LOAD BMP, LOAD JPG, LOAD IMAGE, PLAY MODSAMPLE, PLAY EFFECT, PLAY MODFILE, FRAMEBUFFER and POKE VAR. Their arguments are compiled, and the command itself is prepared once and not again each time it runs. These take the same time as they do in BASIC, because it is the same code doing the work.

What it does not translate

If CC cannot translate a statement it does not stop. The statement is left out, and CC reports the line and the reason. The translation has a comment in its place:

```
/* MMBASIC line 949 not translated: array rank mismatch */
```

Nothing runs in its place, so **read the report**, particularly if the line left out opened a block. A program with a line left out may run, and do the wrong thing.

Some things are refused with an error that names the line:

- LONGSTRING AES128 and the LONGSTRING BASE64 forms;
- STRUCT SORT, SAVE, LOAD, PRINT, EXTRACT and INSERT, STRUCT(FIND), a function that returns a structure, and arrays of structures as parameters;
- an array of structures whose size can change: one that is REDIMmed, or DIMmed with bounds that are not constants (the error is "an array of a TYPE needs constant bounds", at its DIM);
- a GOSUB inside a SUB to a label in the main program;
- RESTART, and the SAVE and RESTORE forms of a few drawing commands.

CC also gives warnings for things that work differently, such as an EXIT with no loop around it. Read them, but a warning does not stop the compile.

Checking the result

A compiled program should behave as the original does. We have compared the output of a 3,200-line calculation byte for byte, and the pictures of two large games pixel for pixel, and they were the same. Your program may do something ours did not, so compare the two: run the BASIC program and the compiled one and check that they agree before you rely on it.

Programs that depend on the time they take can behave differently, because the compiled one is faster. A game that has no frame timer of its own will run too fast.

Very large programs

A Generation 2 has 8MB for the compiler, and a Generation 1 has under 5MB. CC uses smaller tables on a Generation 1 when it has to, and a very large program can then stop with an error such as "jump out of range" or "code segment full". On a Generation 1, a 3,000-line program compiled in 7 seconds and a game of 72KB in about a minute. A game of 122KB, 4,300 lines, needs a Generation 2.

Speed

All of these were measured on a Generation 2 at 480MHz, in MODE 1,8.

Calculation

A 3,200-line program that predicts solar eclipses, run with the three ways of running a BASIC program:

How it ran	Time	Compared with the interpreter
The interpreter	6.16 seconds	1
OPTION COMPILE ON	1.71 seconds	3.6 times faster
CC	0.93 seconds	6.6 times faster

The output of the three was the same, byte for byte, apart from the time the program printed.

The KnivD benchmark moves grains of sand, so a higher figure is better:

How it ran	Grains
The interpreter	18,768
OPTION COMPILE ON	42,070
CC, the BASIC program	69,100
CC, the same program written in C by hand	86,000
The same C built with gcc on a PC, as a loadable program	130,291

Games

Games spend much of their time drawing. That takes the same time however the program runs, so the gain from compiling is smaller:

Game	Interpreter	OPTION COMPILE ON	CC
Wolfenstein 3D (wolf3d_cmm2)	28.5 frames a second	79.7	90.2
Into the Darkness	17.8 frames a second	37.5	37.9

Into the Darkness spends about 62% of its time in the firmware's drawing commands, so compiling the rest gains almost nothing over OPTION COMPILE ON. It is a good example of when OPTION COMPILE ON is all you need.

Which to use

OPTION COMPILE ON needs no extra step and works on any program, so try it first. Use CC when the program is mostly calculation, or when you want the last of the speed and have checked that the result is the same.

Problems

CC says it cannot find the compiler

cc.app has to be in A:/cc. Copy the cc folder from the programs zip to the SD card, and check its name.

"Cannot open include file stdint.h"

The header files in A:/cc/include are from an older version. Copy the whole cc folder from the zip that came with your firmware.

"x.c did not compile"

The lines before it give the file, the line number and the error. If the error is on a line that looks right, look at the line before it: a missing semicolon is reported on the next line. Check the list of what is not supported in the chapter C Programs.

A compiled BASIC program does the wrong thing

Look at what CC reported when it compiled. A line it could not translate is left out, and the program then runs without it. If nothing was reported, run the BASIC program beside it to find where they part, and please report it on the Back Shed forum with the program.

A game runs too fast

The compiled program is faster than the interpreter. A game that paces itself by how long its own code takes needs a timer: wait for the TIMER, or for the next frame, in its main loop.

"not enough memory for its tables", "jump out of range", "code segment full"

The program is too large for the compiler's tables. This is only likely on a Generation 1 (see Very large programs in the chapter MMBasic Programs). Compile it on a Generation 2.

CC seems to be taking a long time

CC prints a row of dots as it compiles. A large program takes a minute or more, and the dots can stop for some time while it works on the largest functions.

The compiled program cannot find its files

A program looks for its files from the current directory, as a BASIC program does. CHDIR to the program's folder before RUN.

Appendix A – Licences

Part	Licence	Where it is
The compiler (cc.app: its passes cc0, cc1 and cc2)	GNU General Public License, version 3	In the cc folder, as a separate program. It is not part of the firmware.
The C preprocessor, cpp	From Fuzix	In cc.app.
The translator from MMBasic to C	The MMBasic licence (Geoff Graham and Peter Mather)	In cc.app.
The runtime of compiled programs, and the BASIC runtime	Not under the compiler's GNU General Public License. The BASIC runtime is under the MMBasic licence.	In the firmware.

The source of all of these is on GitHub, in the repository github.com/UKTailwind/FUZIX, branch cmm2. apps/cc/make.py in the firmware's repository, github.com/UKTailwind/CMM2V1.5, builds them from the commit it names.